
🧭 Project Documentation — Tiipstr Architecture (ArTech)

📘 Overview

Tiipstr is a modern social media application that enables users to give and receive reviews under three distinct profile types:

- **Personal**
- **Professional**
- **Business**

The platform focuses on **trust, transparency, and real-time interaction**, enabling users to connect, share feedback, and manage their reputation in a secure digital environment.

⚙️ Technology Stack

💻 Frontend

- **Library:** React.js
- **Routing:** React Router DOM
- **State Management:** Redux Toolkit
- **Data Fetching:** Axios + TanStack Query (React Query)
- **Realtime Communication:** WebSocket using STOMP.js
- **Responsive Design:** Bootstrap framework
- **KYC Verification:** Cybrid Plaid Integration
- **Payment Gateway:** Authorize.net (Hosted Form Integration)

🧩 Backend

- **Framework:** Spring Boot (Java 17+)
- **Database:** MySQL
- **Migration:** FlywayDB (SQL Migration Scripts)
- **Messaging System:** Apache Kafka (for communication between users/services)
- **Search Engine:** Elasticsearch (for profile and review search optimization)
- **API Documentation & Testing:** Swagger UI / OpenAPI

- **Security:** AWS Cognito Identity Provider (for user authentication and API security)

☁️ Deployment & Cloud Infrastructure

- **Platform:** Amazon Web Services (AWS)
- **Key Services Used:**
  - **EC2 Instances** – for backend and frontend hosting
  - **S3 Bucket** – for file/image storage
  - **AWS Cognito** – for authentication
  - **AWS RDS (MySQL)** – managed database
  - **CloudWatch** – for logging and monitoring
  - **Elastic Beanstalk / ECS (Optional)** – for application deployment & scaling

---

🧱 System Architecture

```
┌──────────────────────────┐
|      Frontend (React)     |
|  - React JS / Redux Toolkit |
|  - Axios / Tanstack Query   |
|  - Bootstrap / Router DOM   |
└─────────────┬─────────────┘
              |
              ▼
┌──────────────────────────┐
|  Backend (Spring Boot)    |
|  - REST APIs (Swagger Docs)|
|  - Flyway Migration       |
|  - Kafka Communication    |
|  - WebSocket STOMP        |
└─────────────┬─────────────┘
              |
```

```
┌───────────────────┼────────────────────┐
│          │          │
▼          ▼          ▼
┌──────────────┐ ┌──────────────┐ ┌──────────────┐
│MySQL Database │ │Elasticsearch │ │AWS Cognito │
│(AWS RDS) │ │(Search Service) │ │(Auth & Security) │
└──────────────┘ └──────────────┘ └──────────────┘
│
▼
┌────────────────────┐
│AWS S3 (File Storage) │
└────────────────────┘
```

Additional Integrations:

- Plaid (KYC)

- Authorize.net (Payments)

🚀 Key Modules

1. User Module

- Register/Login using AWS Cognito
- Manage personal/professional/business profiles
- KYC verification via Cybrid Plaid

2. Review Module

- Give or receive reviews between users
- Manage review categories (personal, professional, business)
- Realtime updates using WebSockets (STOMP.js)

3. Feed / Social Module

- Display user reviews and interactions in real-time

- Kafka handles event streaming between active users

4. Search Module

- Elasticsearch powers advanced search and filtering of profiles and reviews

5. Payment Module

- Payment processing via Authorize.net hosted form
- Secure transaction handling and reporting

6. Admin Module

- API documentation and testing via Swagger
- Database versioning via FlywayDB

🔐 Security & Authentication

- **AWS Cognito** handles user registration, authentication, and token management.
- **HTTPS / SSL** enforced across all endpoints.
- **JWT (JSON Web Tokens)** used for session handling.
- **Role-Based Access Control (RBAC)** implemented in backend.

🔄 Data Flow

1. **Frontend** calls backend APIs via **Axios** or **Tanstack Query** for optimized caching.
2. Backend processes request → interacts with **MySQL** for CRUD operations.
3. **Kafka** streams events (notifications/messages) between microservices.
4. **Elasticsearch** provides instant search results.
5. Realtime updates pushed to frontend using **WebSocket (STOMP.js)**.
6. Files/media stored in **AWS S3**.
7. Secure authentication/authorization handled via **Cognito**.

🧰 Developer Tools

- **VS Code / IntelliJ IDEA** – IDEs
- **Postman** – API Testing

- **Swagger UI** – Live API docs
- **GitHub / GitLab** – Version Control
- **Maven / npm** – Build Tools
- **AWS Console** – Cloud management

Deployment Flow

1. **Code Commit → GitHub**
2. **Build (Maven / npm run build)**
3. **Deploy Frontend** → AWS S3 or CloudFront
4. **Deploy Backend** → AWS EC2 / Elastic Beanstalk
5. **Database Migration** → Flyway executes SQL scripts
6. **Monitor Logs** → AWS CloudWatch

Performance & Optimization

- **Tanstack Query** for efficient data caching and revalidation.
- **WebSocket STOMP.js** for real-time updates.
- **Kafka** ensures event-driven, asynchronous user communication.
- **Elasticsearch** enables fast, full-text searching.
- **AWS Auto Scaling** ensures stability under heavy traffic.

Future Enhancements

- Integration with AI-driven sentiment analysis for reviews.
- Push notifications using AWS SNS.
- Multi-language support.
- Advanced analytics dashboards.
